

TECHNICAL SPECIFICATION & CODEBASE REVIEW

chi-vercel

A single Express server hosting a portfolio of independent mini-apps and games, plus three MySQL-backed JSON APIs

Scope: full repository — main server, 11 mounted sub-applications, dependencies, deployment configuration

Method: derived directly from the current source tree (no design docs existed to draw from)

Generated: 29 August 2026

Contents

1. Executive Summary
2. Architecture Overview
3. Main Server — Routes & Functions
 - 3.1 Page & static routes
 - 3.2 Generic upload endpoint
 - 3.3 Albums API
 - 3.4 Playlist API
4. Sub-Applications
5. Libraries & Dependencies
6. Findings — Bugs, Risks & Discrepancies
7. Environment & Deployment

1. Executive Summary

chi-vercel is a single Node.js/Express process (`index.js` , ~640 lines, no framework abstraction beyond Express itself) that does two jobs at once:

1. Serves a personal **portfolio hub** at `/ / /chi` — a simple EJS page listing links into 11 small, fully independent client-side sub-applications, each mounted as its own `express.static` directory.
2. Hosts three small **JSON APIs** (`/api/albums` , `/api/albums-new` , `/api/playlist`) plus two file-upload endpoints, all backed by a single remote MySQL database and a single remote FTP account on HostGator — there is no local/dev database, no ORM, and no connection pooling (each request opens and closes its own `mysql.createConnection`).

There is no test suite (`npm test` runs `node test.js` , but `test.js` does not exist), no linter, no formatter, and no CI configuration in the repository. Each sub-app is a self-contained artifact — some hand-written vanilla JS, some Vue 2 loaded from a vendored script with no build step, some React loaded via CDN + in-browser Babel with no build step, and some modern Vite-built React bundles — checked directly into the repo with no shared frontend tooling or monorepo setup between them. This is a deliberate, low-overhead structure appropriate for a collection of small personal projects, but it means every sub-app must be reasoned about independently.

The codebase review surfaced several concrete issues, detailed in full in [Section 6](#): a SQL-injection surface across the entire playlist API, an error-handling bug that causes some playlist requests to hang instead of responding, a broken Vercel deployment configuration (two sub-apps would 404 in production), an undeclared direct dependency (`body-parser`) that only works by accident of dependency hoisting, and a completely unused dependency (`pg`). It also found that the project's own `CLAUDE.md` guidance file is out of date on two points — one pleasantly (a previously-documented Vercel misconfiguration has since been fixed) and one not (the `playlist` sub-app has been rewritten from Vue 2 to React since that file was last updated, and it no longer says so).

2. Architecture Overview

Process & entry point

- Single process, single port: `process.env.PORT || 5000`.
- `npm start` → `node index.js`. Node engine pinned to `24.x` in `package.json#engines` (the local dev shell used for this review was on Node 16/18 via `nvm` — worth confirming the deploy target actually runs 24.x, since nothing in the repo enforces it locally).
- `index.js` both builds the Express app *and*, when run directly (`require.main === module`), starts listening — it also exports the bare `app`, which is what a serverless platform (Vercel) would import instead of calling `.listen()`.

Static mounts

URL prefix	Filesystem folder	In <code>menu.js</code> (linked from <code>/chi</code>)?
<code>/albums</code>	<code>albums/</code>	Yes
<code>/playlist</code>	<code>playlist/</code>	Yes
<code>/itunes100</code> , <code>/itunes100/:country</code>	<code>itunes100/</code>	Yes
<code>/bbb</code>	<code>bbb/</code>	Yes
<code>/citadels</code>	<code>citadels/</code>	Yes
<code>/poker-chips-counter</code>	<code>poker-chips-counter/</code>	Yes
<code>/m-combinations</code>	<code>m-combinations/</code>	No — direct URL only
<code>/mafia-random-number</code>	<code>mafia-random-number/</code>	No — direct URL only
<code>/mafia-random-card</code>	<code>mafia-random-card/</code>	No — direct URL only
<code>/superfrog</code>	<code>superfrog/</code>	No — direct URL only
<code>/metromap-builder</code>	<code>metromap-builder/</code>	No — direct URL only

Root-level `public/` is also statically served (shared hub assets — logo, hub stylesheet).

View layer

EJS (`view engine = ejs`, views in `views/`). Only two pages are actually rendered: `pages/chi.ejs` (the hub, fed the `menu` array from `menu.js`) and `pages/upload.ejs` (a bare file-picker that `fetch()` s `POST /upload-new`). `views/partials/nav.ejs` and `views/partials/header.ejs` exist on disk but are not `include` d anywhere — see [Finding F-7](#).

Outbound integrations

Everything database- or file-storage-related points at one shared HostGator account:

- **MySQL** — host `gator4255.hostgator.com`, database `chitwofo_playlist`, user `chitwofo_247`, password from `process.env.HOSTGATOR_PWAD`. A brand-new `mysql.createConnection()` is opened and torn down per request in every handler — there is no pool, so each API call pays a full TCP + MySQL handshake.
- **FTP** — same host, user `chitwofo`, same `HOSTGATOR_PWAD` password (the DB and FTP account intentionally share one secret). Used to push album cover images to `/public_html/albums-covers/`.
- Mutations (`add` / `edit` / `delete` on albums and playlist deletion) are gated by a single shared password, `process.env.EDIT_PWAD`, posted in the request body and compared with plain `!=`.

Because both of these require real production credentials, none of the DB/FTP-touching endpoints can be exercised in a local/dev environment.

3. Main Server — Routes & Functions

All of the following live in the single file `index.js`.

3.1 Page & static routes

Route	Method	Behaviour
<code>/</code> , <code>/chi</code>	GET	Renders <code>pages/chi.ejs</code> with the <code>menu</code> array — the portfolio hub.
<code>/upload-file</code>	GET	Renders <code>pages/upload.ejs</code> — a bare file picker.
<code>/cool</code>	GET	Returns a random string from the <code>cool-ascii-faces</code> package. No other logic.

3.2 Generic upload endpoint

`POST /upload-new`

Accepts one `multipart/form-data` file field named `upload_file` (parsed with `formidable`) and FTPs it, **under its original filename, unmodified**, to `/public_html/albums-covers/` on HostGator. Responds `{ ok: true }` or `{ error: 'Error file uploading' }`. Used by the standalone `/upload-file` page — a generic sibling to the cover-upload path baked into `/api/albums-new` below.

3.3 Albums API

POST /api/albums-new — multipart form, full CRUD

Body field `action` selects behaviour:

action	Auth	What it does
<code>list</code>	—	<code>SELECT</code> from <code>albums</code> , optionally filtered by <code>year</code> / <code>month</code> (only when both parse as positive integers).
<code>add</code>	<code>pass === EDIT_PWAD</code>	If a <code>cover</code> file was posted, FTPs it to <code>/public_html/albums-covers/</code> under the name <code>\${year}_\${month}_\${Date.now()}</code> , then <code>INSERT</code> s the row. All text fields are escaped only for the single-quote character (<code>replaceAll("'", "''")</code>) before interpolation.
<code>edit</code>	<code>pass === EDIT_PWAD</code>	Same cover-upload behaviour as <code>add</code> , then <code>UPDATE ... WHERE id=\${id}</code> — <code>id</code> itself is interpolated with no validation or escaping.
<code>delete</code>	<code>pass === EDIT_PWAD</code>	<code>DELETE FROM albums WHERE id=\${id}</code> — same unvalidated <code>id</code> interpolation.

POST /api/albums — JSON body, read/delete only

A reduced, JSON-bodied sibling of the above used by the albums client for the read and delete paths only (actions `list` / `delete`); `add` / `edit` are not implemented here.

3.4 Playlist API

POST /api/playlist — one large action switch

Backs a weekly-chart / radio-playlist tracker across three tables: `songs`, `playlist` (one row per song per playlist date, carrying a `score`), and `bonuses` (a per-year, per-month-cutoff end-of-year bonus schedule).

action	Purpose
<code>current</code> / <code>latest</code>	Resolve to a playlist date (either the most recent on/before <code>current_date</code> , or the global latest) and return its full song list, ordered by score then artist.
<code>prev</code> / <code>next</code>	Step to the adjacent playlist date relative to <code>pl_date</code> .
<code>previous</code>	Jump to the nearest playlist date at least 7 days before <code>pl_date</code> (i.e. "same slot, prior period").
<code>top100</code>	Year-to-date top 100 songs by summed score plus a per-year end-of-year bonus (see below). Lazily seeds that year's bonus schedule into <code>bonuses</code> on first use via <code>check_bonus</code> .
<code>top10artists</code>	Same underlying ranking, re-aggregated to the top 10 artists by summed total.
<code>upload</code>	Parses a pasted plain-text "Playlist" dump (see domain model below), rejects duplicate dates, upserts each song (matching by exact artist+title), inserts one <code>playlist</code> row per song, and stamps <code>songs.date_appear</code> for any song marked new with a leading <code>*</code> .
<code>delete</code>	Password-gated (<code>EDIT_PWAD</code>). Deletes all <code>playlist</code> rows for a date, then deletes any <code>songs</code> rows that only ever appeared on that one date (i.e. cleans up orphaned songs).

Upload domain model

The pasted dump is split into three fixed-size, fixed-offset blocks with different scoring:

Block	Line offset	Count	Score per song
A	4	9	47
B	16	10	28
C	29	6	23

A leading `*` on a song line marks it new. The header line (index 1) is parsed by `getPlaylistDate`, which expects the fixed-width format `"Playlist DD Month YYYY"` (it slices from character 8 onward and maps the English month name to a two-digit number).

The year-end bonus schedule (seeded once per year, twelve rows keyed by a month-end cutoff date) is: 150, 135, 120*, 105, 90, 75, 150, 120, 90, 60, 30, 0 — applied to the months Jan–Dec in order. * February's cutoff is the 28th or 29th depending on `isLeapYear(year)`.

4. Sub-Applications

Each of the following is a fully self-contained client (own framework, own build artifacts checked directly into the repo) that the Express server does nothing but host as static files.

Albums Catalog [/albums](#) [linked](#)

React (Create React App production build — `static/js/*.chunk.js`). Browsing a catalog of music albums grouped by year/month; calls `POST /api/albums` for listing and deleting, and `POST /api/albums-new` (a separate client bundled under `albums-new` conventions per the server) for adding/editing entries with an optional cover-image upload.

Playlist [/playlist](#) [linked](#)

React, built with Vite (`assets/index-*.js`, ES module bundle — confirmed via React fiber markers in the bundle, zero Vue markers). Displays and navigates the weekly playlist/chart data and its computed year-end top 100 / top 10 artists, driving every action of the `/api/playlist` endpoint described above.

iTunes Top 100 [/itunes100](#) [linked](#)

Plain vanilla JS (`main.js`, no framework, no build step). Fetches Apple's public *topsongs* RSS/JSON feed directly from the browser via a dynamically-injected `<script src="https://itunes.apple.com/<country>/rss/topsongs/.../json?callback=parseResponse">` (JSONP-style, working around CORS) for one of 7 supported storefronts (UK, USA, Australia, Canada, New Zealand, Ireland, Russia). Renders the top-100 table with inline 30-second audio previews that auto-advance to the next track. This app makes no calls back to the chi-vercel server at all beyond loading its own static assets.

Citadels Game [/citadels](#) [linked](#)

React, built with Vite. A client-only digital implementation of the *Citadels* board game; makes no calls back to the server.

Balls Break Bricks

[/bbb](#)[linked](#)

Custom breakout/arcade game built on **Crafty.js** (vendored `crafty-min.js` + `chroma-min.js` for score-tier colour gradients), entirely client-side. Falling numbered bricks descend a row at a time each round; balls bounce to break them, extra balls can be picked up mid-fall, and the run ends if a brick reaches the floor. Features added/hardened directly in this codebase (not part of the original template): a start screen with a `GOTO<n>` cheat-code level-jump (levels 2–400), a single "last cleared level" checkpoint used to resume after death, and a full autosave/restore of in-progress game state to `localStorage` so an accidental page refresh resumes exactly where play left off.

Superfrog

[/superfrog](#)[not in menu.js](#)

A side-scrolling platformer built with **Phaser 3.70** (loaded from CDN, no build step, non-module plain scripts), styled after Team17's *Superfrog*. All art and sound are generated procedurally at boot — tiles/sprites via Phaser `Graphics → generateTexture()`, SFX via a small Web Audio oscillator synth — so the app ships with zero binary art/audio assets. Four levels (1–1 "Forest" through 1–4 "Boss Gate"), each defined as pure data (a walkable height-map plus an entity spawn list) in `src/level.js`, independent of any Phaser calls — the same shape a real Tiled JSON export would produce, making that a drop-in future replacement. Level 1-4 ends in a boss fight: a multi-hit enemy that must be stomped three times before the level-end flag is revealed. Progress (current level / checkpoint) round-trips through `GameOverScene` back into `MainScene` via scene-start data, not persisted storage.

M-Combinations

[/m-combinations](#)[not in menu.js](#)

Vue 2 (vendored `vue.js`, options API, no build step). A companion utility for the *Mafia* party game: given a player count and a target sub-group size `m`, computes every valid `k`-combination of player numbers, filtering out combinations that violate colour-coding rules (a "red" player excludes the whole combination; a set count of "black" players must all be included together) and any combination containing a recorded "contra" (mutually exclusive) pair. State (`playersCnt`, `m`, `players`, `contras`) is persisted to `localStorage` on `beforeunload` / `pagehide` and restored on load.

Mafia Random Number

[/mafia-random-number](#)[not in menu.js](#)

Vue 2 (vendored, inline `<script>` in `index.html`, no build step). Generates a shuffled reveal order for a given player count and steps through it one player at a time — a simple "randomize turn order" utility for the same party game.

Mafia Random Card

`/mafia-random-card`

not in menu.js

React 16 + Redux + `react-redux`, all loaded via CDN `<script>` tags (`unpkg` / `cdnjs`) with **in-browser Babel-standalone JSX transpilation** — i.e. no build step at all; the `.js` component files are served and transpiled on the fly by the browser. State is a small Redux store (`store.js` + `actions.js`: player/mafia counts, extra special-role cards toggled in/out, per-player sequential dealing). Deals a random role card per player from a sprite sheet, persists the in-progress session to `localStorage` the same way as M-Combinations (`beforeunload` / `pagehide`).

Poker Chips Counter

`/poker-chips-counter`

not in menu.js

React, built with Vite. A poker chip buy-in/denomination calculator; client-only, no server calls.

Metro Map Builder

`/metromap-builder`

not in menu.js

React, built with Vite. A metro/subway-style diagram builder; client-only, no server calls. The newest addition to the repo (also missing from `vercel.json`'s deploy allowlist — see [Finding F-2](#)).

5. Libraries & Dependencies

Server (declared in `package.json`)

Package	Version	Used for
<code>express</code>	<code>^4.15.2</code>	HTTP server, routing, static file serving.
<code>ejs</code>	<code>^3.1.5</code>	Server-rendered hub (<code>/chi</code>) and upload pages.
<code>formidable</code>	<code>^2.0.1</code>	Multipart/form-data parsing for all file-upload endpoints.
<code>ftp</code>	<code>^0.3.10</code>	Pushing album cover images to HostGator.
<code>mysql</code>	<code>^2.18.1</code>	All albums/playlist database access — a new connection per request, no pool.
<code>cool-ascii-faces</code>	<code>^1.3.4</code>	Backs the standalone <code>GET /cool</code> route only.
<code>pg</code>	<code>^8.7.3</code>	unused Not required or referenced anywhere in the codebase — see Finding F-4.

Dev dependencies

Package	Version	Status
<code>tape</code>	<code>^4.7.0</code>	unused No test file exists to use it.

got	^11.3.0	unused Same — not referenced anywhere.
-----	---------	--

Undeclared but required

Package	Status
body-parser	not in package.json Directly require() d in index.js but absent from dependencies — see Finding F-3.

Sub-application frontend stacks

App	Stack	Build
Albums Catalog	React	Create React App (production chunks checked in)
Playlist	React	Vite (bundle checked in)
iTunes Top 100	Vanilla JS	None
Citadels Game	React	Vite (bundle checked in)
Balls Break Bricks	Crafty.js (vendored) + chroma.js (vendored)	None
Superfrog	Phaser 3.70 (CDN)	None — plain scripts
M-Combinations	Vue 2 (vendored)	None
Mafia Random Number	Vue 2 (vendored)	None
Mafia Random Card	React 16 + Redux + react-redux + styled-components (all CDN) + Babel-standalone	None — in-browser JSX transpile
Poker Chips Counter	React	Vite (bundle checked in)
Metro Map Builder	React	Vite (bundle checked in)

None of the sub-apps have their own `package.json` in this repo — each is a pre-built artifact (or, for the no-build ones, source files) checked in directly, with no shared node_modules or build tooling between them.

6. Findings — Bugs, Risks & Discrepancies

F-1 • SQL injection surface across the entire Playlist API

Every query in the `/api/playlist` handler is built by direct string interpolation with **zero** escaping — `year`, `pl_date`, `current_date`, and every field parsed out of the uploaded playlist text all flow straight into SQL. (The Albums API is somewhat better off: it escapes single quotes on text fields, but `id` is still interpolated unescaped into `UPDATE` / `DELETE` statements.) This is exploitable by anyone who can reach the endpoint, regardless of the `EDIT_PWAD` gate on mutating actions — read-only actions like `current` / `top100` are just as vulnerable.

F-2 • Vercel deployment is broken for two sub-apps

`vercel.json`'s `includeFiles` allowlist is:

```
{albums,bbb,citadels,itunes100,m-combinations,mafia-random-card,mafia-random-number,playlist,poker-chips-counter,public,views}/**
```

This omits `superfrog` and `metromap-builder`. Both are correctly `express.static`-mounted in `index.js` and work fine locally, but on an actual Vercel deploy using this config their static files would not be bundled into the serverless function, and both routes would 404 in production.

F-3 • `body-parser` is used but not a declared dependency

`index.js` does `const bodyParser = require('body-parser')`, but `package.json#dependencies` does not list it. It currently resolves only because `express` pulls it in transitively and npm happens to hoist it to the top-level `node_modules/`. This is fragile — a stricter install (or a future Express major version that changes or drops that transitive dependency) would break the app with no warning from `package.json` itself.

F-4 • Unused dependencies

`pg` (a full PostgreSQL client) is declared but never required anywhere in the codebase — dead weight, or a leftover from an abandoned migration off MySQL. `tape` and `got` (devDependencies) are likewise never referenced, consistent with `test.js` not existing.

F-5 • Playlist upload/delete: inner callback shadows `res`, silently swallowing errors

Inside `insertPlaylistItem`:

```
con.query(ins_query, (err, res) => {
  if (err) {
    con.end()
    res.end(JSON.stringify({ error: 'Error adding playlist'})) // res is the MySQL
    result, not Express's res
    return
  }
  ...
})
```

The callback parameter `res` shadows the outer Express response object. On the error path, `res.end(...)` is called on the **MySQL query result**, not the HTTP response — which has no `.end()` method, so this throws inside an async callback with no surrounding try/catch. The net effect: if an insert fails partway through an `upload`, the client's request simply hangs (no response ever sent) until it times out, and the count-based completion tracker (`count_inserted`) never reaches its target either, so a partial upload can't be detected or retried cleanly by the caller.

F-6 • Shared, non-constant-time password check

All mutating actions across both APIs (albums add/edit/delete, playlist delete) are gated by one plaintext secret (`EDIT_PWAD`) posted in the request body and compared with `!== / ===`. There is no rate limiting, no per-user credentials, and the comparison is not constant-time (a minor timing-attack surface on top of the bigger issue of a single shared secret protecting every destructive action across two unrelated APIs).

F-7 • Dead view partials

`views/partials/nav.ejs` and `views/partials/header.ejs` are not `include`d by any page (only `chi-header.ejs` and `upload-header.ejs` are). Their content — Bootstrap glyphicon nav, "How Heroku Works" links, a Getting Started dropdown — reads like leftover scaffolding from the original Heroku Node.js starter template this project was bootstrapped from.

N-1 • `CLAUDE.md` is stale on two points

Already fixed, doc not updated: `CLAUDE.md` says `vercel.json` "declares a build for `api/index.js`", but the actual entry point is `./index.js`. That mismatch no longer exists — `vercel.json`'s `builds[0].src` is now literally `"index.js"`, matching the real entry point (see [Finding F-2](#) for the deploy config's *actual* current problem, which is the missing folders, not the entry-point path).

Not yet reflected in the doc: `CLAUDE.md` describes `playlist/` as "hand-written Vue 2 + vendored libs". The currently checked-in build (`playlist/assets/index-*.js`, a Vite ES-module bundle carrying React fiber markers and no Vue markers) is a React app. The app's files are dated after `CLAUDE.md`'s own last edit, so it appears to have been rewritten since that guidance was written.

7. Environment & Deployment

Required environment variables

Variable	Used for
<code>HOSTGATOR_PWAD</code>	Shared password for both the HostGator MySQL database (<code>chitwofo_playlist</code> , user <code>chitwofo_247</code>) and the HostGator FTP account (<code>chitwofo</code> @ <code>gator4255.hostgator.com</code>).
<code>EDIT_PWAD</code>	Gate password required in the request body for any mutating action (<code>add</code> / <code>edit</code> / <code>delete</code>) on the albums or playlist APIs.
<code>PORT</code>	Optional; defaults to <code>5000</code> for local/Heroku-style hosting.

Running locally

```
npm install
npm start      # node index.js, listens on $PORT or 5000
```

Endpoints that don't touch MySQL/FTP (the hub page, and every sub-app that makes no server calls — see Section 4) work with no environment variables at all. Anything hitting `/api/albums*` or `/api/playlist` needs real production HostGator credentials; there is no local database or FTP stand-in.

Deployment targets

- **Heroku-style (or any Node host):** works as-is via `npm start` — this is the primary supported path.
- **Vercel:** configured via `vercel.json` (single serverless function wrapping `index.js`, CORS headers applied to every route), but currently broken for two sub-apps — see Finding F-2.

Generated by inspecting the repository directly — `index.js`, every sub-app's entry point and build output, `package.json` / `package-lock.json`, `menu.js`, `vercel.json`, and the EJS view layer. No external

documentation existed to draw from.